Portfolio Project:

Software Development Ethical Considerations - 23andMe Case Study

Alexander Ricciardi

Colorado State University Global

CSC502: Ethical Leadership in Software Development

Professor: Dr. Steven Evans

March 8, 2026

Portfolio Project:

Software Development Ethical Considerations - 23andMe Case Study

Software systems are not merely technical artifacts. They are institutions in miniature: they set rules about what can be collected, who can see it, how long it is retained, and what happens when an organization changes ownership or fails. Those rules may be implemented in code, but they are ethical decisions in practice, because they shape real risks and real harms for individuals and communities. The ethical stakes are especially high for systems that process genetic information. Genetic data is unusually sensitive because it is persistent, uniquely identifying, and informative not only about the individual who submitted a sample but also about biological relatives (National Library of Medicine, 2022). In Direct-To-Consumer (DTC) genetic testing, many core product features, including matching relatives, ancestry composition, health reports, and optional research participation, make the data valuable to users but also amplify consequences if security controls, defaults, or governance are weak.

This paper uses 23andMe as a case study because it illustrates how common security failures and ambiguous governance choices can cascade into large-scale ethical risk. Canadian and UK privacy regulators concluded that 23andMe was subject to a lengthy credential-stuffing attack (April-September 2023), which resulted in access to 18,222 accounts and, by leveraging the DNA Relatives feature, exposure of information associated with nearly 7 million additional customers (Office of the Privacy Commissioner of Canada, 2025). The same regulator findings also emphasized detection gaps and communication failures, including the organization's delayed recognition of the attack and shortcomings in breach notifications. A second layer of ethical concern emerged during bankruptcy.

In March 2025, 23andMe filed for Chapter 11 bankruptcy, which raised public anxiety that sensitive genetic information could be sold or transferred as part of the bankruptcy process. The Federal Trade Commission (FTC) warned that any purchaser must adhere to the company's existing privacy promises and emphasized consumer concern about data transfers in bankruptcy (Ferguson, 2025). In June 2025, New York's Attorney General and a coalition of other attorneys general argued that 23andMe could not auction genetic data without customers' knowledge or consent (New York State Office of the Attorney General, 2025). Taken together, the breach and the bankruptcy context illustrate why software ethics must include both technical safeguards and organizational governance. To support a rubric-level analysis, the paper first identifies concrete ethical lapses and then answers the assignment prompts using established professional and regulatory frameworks. In particular, it draws on the Association for Computing Machinery (ACM) Code of Ethics (ACM, 2018) and the National Institute of Standards and Technology (NIST) Privacy Framework (NIST, 2020) to convert ethical principles into implementable engineering and governance practices.

Ethical Lapses Identified in the 23andMe Case

Ethical analysis becomes more actionable when it is anchored to specific failures and design choices. Based on regulator findings, company disclosures, and public reporting, the 23andMe case highlights at least six ethical lapses that are common in data-intensive software systems. 23andMe's first lapse was not having adequate protection against credential-stuffing and account takeover.

An investigation from the Office of the Privacy Commissioner of Canada (2025) concluded that 23andMe experienced a credential-stuffing attack for months, and that signals (such as distorted success/failure login ratios) could have helped with earlier detection. 23andMe's incident response action plan also described the incident as a credential stuffing attack and recommended enabling Multi-Factor Authentication (MFA) (23andMe, 2023). The investigation also found that a product exposure lapse was amplified by the software's social matching features. Although most of these features were user opt-in options, it was 23andMe's ethical responsibility to assess whether their design choices could compromise user accounts and cause harm at massive scale. Another lapse was the inability of 23andMe to identify and respond to the credential-stuffing attack until October 2023 due to insufficient monitoring, attack detection, and incident-response. Although the company was aware of earlier platform instability and unusual activity, 23andMe addressed the breach only after the stolen data was advertised for sale. This raises ethical questions about whether the prolonged signals were treated as routine technical noise rather than indicators of possible harm. Additionally, the joint findings from the commission investigation and 23andMe's own investigation express concerns about the organization's completeness and timeliness of notification lapses. It was found that the methodology of identifying affected individuals, especially high-sensitivity cases, such as raw illegal DNA downloads, was overly narrow, and an implementation of broader analysis could have helped identify more impacted accounts. Another major lapse was the weak clarity about what happens to the data during business transitions, especially the sale of the organization after bankruptcy.

The FTC letter addressed this explicitly by acknowledging consumer fear about the sale or transfer of sensitive genetic data during bankruptcy and emphasized that privacy promises must be honored (Ferguson, 2025). From an ethics standpoint, data governance after bankruptcy and acquisition should not be treated as an edge case; rather, it should be treated as a part of the data lifecycle and responsible stewardship, especially for highly sensitive data. Finally, 23 consent and data reuse ambiguity lapse represents a critical breakdown in user trust. With DTC genetic testing, ethical concerns have consistently been raised about how clearly customers understand secondary uses (such as research or partnerships) and whether consent is informed. Research published in the Journal of Personalized Medicine found that consumers often face complex terms and uncertainty about privacy expectations in DTC genetic testing (Hendricks-Sturrup & Lu, 2019). Furthermore, Reuters reported that 23andMe had made at least 30 deals with companies (including GSK); moreover, many agreements were not publicly disclosed, and users were urged, out of fear, to delete 23andMe (Satija & Mahatole, 2025). Ethically, this lack of transparency validates the public concern about consent and future data use during bankruptcy, especially regarding the commercialization and reuse of DNA data. These lapses are not simply bugs or security mistakes. They are failures of ethical responsibilities; organizations and software developers have the obligation to anticipate harms, design safe features, monitor for abuse, and communicate transparently when the harms resulting from risks materialize.

What Makes Ethics Important for Software Developers?

In software development, ethics is essential because software mediates power as it balances functionality and risk. The decision on what data to collect, how features are designed, and what security is required is often made solely by software developers with little involvement from outside stakeholders.

This means that developers indirectly decide how much harm is acceptable as a trade-off for convenience, growth, or revenue. The ACM Code of Ethics defines this principle as computing professionals' ethical responsibility to promote human well-being and avoid harm, including harms caused by disclosure of information (ACM, 2018). The Code defines harm as the negative, detrimental consequences resulting from the purposeful, negligent, or autonomous use of digital technologies. This implies that harm is not limited to financial loss, but it also includes privacy and reputational damage, psychological distress, discrimination, and erosion of trust.

In the context of genetic data, this ethical responsibility is intensified for two main reasons. First, due to the nature of DNA information being persistent and uniquely identifying, it is difficult or impossible to reset genetic data once leaked. Second, genetic information is relational, meaning that the data can reveal information about family members who did not consent to data sharing or analysis (National Library of Medicine, 2022). This implies that the ethical responsibility of collecting, managing, and processing DNA data goes beyond direct users to non-users who may still be impacted; this highlights that, in the context of ethics, compliance is not the same as legitimacy. In other words, software developers can build software systems or applications that comply with regulations and are technically legal, and still not fulfill their ethical responsibilities by violating the principles of fairness, clarity, and respect for persons. For example, a user consent that is buried in dense small print contracts or agreements may be technically lawful, but it does not necessarily ensure that the user understands and is meaningfully informed. The research published in the Journal of Personalized Medicine showed that the terminology of DTC genetic testing is not fully understood by consumers, creating concerns about privacy expectations and reuse of the data collected from the testing (Hendricks-Sturrup & Lu, 2019).

For developers, this matters because software is increasingly embedded in decision-making, influencing how data is interpreted and applied. Especially when software is combined with Artificial Intelligence (AI) and data science, model training, and analytics, such as the inference of traits, profiling, and impacts. Furthermore, this data processing risk is emphasized when data is unique and persistent, such as DNA information. Ultimately, professional ethical responsibility requires developers to treat privacy, security, and governance as part of software quality, not as after-the-fact compliance work.

Examples of Ethical Considerations Developers Might Face

The 23andMe case analysis showcases perfectly the ethical considerations that developers face when designing, implementing, and monitoring software systems. For example, DTC genetic testing involves managing and processing personal information, including health-related data, ancestry information, and potentially raw genetic data (Office of the Privacy Commissioner of Canada, 2025). When handling such data, developers must decide what is stored, what is shared, and under what permissions. They need to recognize that a design prioritizing functionality over security is an ethical choice; in other words, a software that is useful but does not effectively protect privacy risks exposing data, which the ACM code categorizes as harm (ACM, 2018). User consent needs to be also considered not just as a legal checkbox procedure, that is, whether consent exists, but whether it is understandable. The National Library of Medicine emphasizes that consumers should read and understand privacy information before using DTC genetic testing and provides a set of questions for evaluating privacy practices (National Library of Medicine, 2022).

However, the research published in the Journal of Personalized Medicine also suggests that DTC genetic testing policies can be difficult for consumers to interpret, undermining informed consent (Hendricks-Sturrup & Lu, 2019). Requiring MFA, implementing strict rate limits, or adding verification steps increases security but may reduce usability and conversion. A possible solution to these issues is implementing privacy-by-design principles as a core functional requirement, and prioritizing safeguards such as MFA, implementing strict rate limits, or adding verification steps, which increases security but may reduce usability and conversion.

Ultimately, the core consideration that software developers face is whether convenience is prioritized over foreseeable harm. In the 23andMe case, the Office of the Privacy Commissioner of Canada (2025) investigation concluded that MFA and calibrated detection for credential stuffing, if implemented by 23andMe, would have protected against the credential stuffing attack. This lapse is an example of convenience prioritization over mitigating foreseeable harm. Furthermore, it is amplified by the nature of genetic information, as it can be used to discriminate. Although regulations exist against discrimination based on genetic information, such as the Genetic Information Nondiscrimination Act (GINA) prohibit employers from using genetic information in hiring, firing, or promoting employees, leakage of genetic information can still lead to stigma, family conflict, or insurance-related issues (Equal Employment Opportunity Commission, 2014; Panacer, 2023). This reinforces the need for software developers to uphold their ethical responsibility fully rather than treating ethics as merely a process for legal compliance.

What Can Developers Do to Incorporate Ethical Considerations into Their Work?

Ethical software development requires both professional commitments and operational practices. In practice, this means embedding ethics into requirements, architecture, testing, monitoring, and governance, not treating it as a final policy review. Two frameworks are especially helpful for turning values into repeatable actions: the ACM Code of Ethics (ACM, 2018) and the NIST Privacy Framework (NIST, 2020). The ACM Code provides a professional baseline: avoid harm, respect privacy, be honest and trustworthy, and take action when systems threaten harm. In engineering terms, this supports concrete practices such as:

- Threat modeling for harm, not only for system compromise (e.g., What is the worst plausible harm if an account is taken over?).
- Designing safer defaults (e.g., limiting exposure of sensitive profile fields).
- Escalation pathways that empower engineers to raise concerns when risk is being ignored.

NIST's Privacy Framework is explicitly designed to help organizations manage privacy risk through enterprise risk management. This can be used as an implementation map that connects privacy outcomes to engineering controls. The five NIST Privacy Framework functions, Identify-P, Govern-P, Control-P, Communicate-P, and Protect-P can be translated into development activities as follows:

- Identify-P (know what data exists and where), maintaining accurate data inventories; classify genetic/health data as high-sensitivity; document data flows for features such as DNA Relatives.

- Govern-P (set accountability and policy), assigning owners for privacy/security decisions; define acceptable risk thresholds; require review for changes in sharing defaults and external data uses.
- Control-P (enable user agency and limitation), providing meaningful consent controls; ensure deletion, revocation, and opt-out mechanisms are usable and effective.
- Communicate-P (clarity and transparency), providing plain-language explanations of data use, sharing, and breach response; communicate changes during bankruptcy or acquisition.
- Protect-P (security safeguards), applying strong authentication, anomaly detection, rate limiting, logging, and breach response capabilities.

In the 23andMe case, regulators recommended safeguards against credential-stuffing, such as MFA and calibrated detection for suspicious activity (Office of the Privacy Commissioner of Canada, 2025). This recommendation is an example of how Protect-P (security) must be paired with Identify-P (understanding data exposure) and Communicate-P (clear user guidance and notifications). Another important action a developer can take to incorporate ethics within their professional practice is to incorporate ethical review directly into the Software Development Life Cycle (SDLC). Within requirements, treating privacy and security as non-functional requirements. Within the software architecture, by limiting blast radius, such as avoiding designs where one compromised account enables access to thousands of other users' profiles. Within the software testing component, by implementing abuse-case tests, such as rate-limiting, scraping resistance, and account takeover simulations. Within the software monitoring process, by implementing detection measures for credential stuffing and automated scraping processes. Finally, within the incident response plan, by having and maintaining

playbooks and ensuring that if breaches occur, their notifications are accurate and detailed. Implementing these integrated safeguards aligns with quality, reliability, and risk management best practices and upholds developers' ethical responsibility as a core discipline of software development rather than treating it as a secondary task.

What Type of Questions Should Developers Be Asking?

To help with the ethical consideration process, software developers may ask a set of questions to evaluate the ethical impact of their technical choices, especially during design decisions where tradeoffs are easy to rationalize. The following set of questions, based on the 23andMe case, can be used as examples to test the ethical compliance of their technical choices:

1. What is the minimum data needed to provide the service, and what can we avoid collecting or storing?
2. Are defaults privacy-protective, especially for social/sharing features that implicate relatives?
3. Could common attacks (scraping, credential stuffing, automated enumeration) lead to large-scale access, and do we have layered defenses such as MFA, rate limits, and anomaly detection (Office of the Privacy Commissioner of Canada, 2025; 23andMe, 2023)?
4. Can users clearly understand, in plain language, who might access their data, whether data may be used for research or partnerships, and how deletion and consent withdrawal work (National Library of Medicine, 2022; Hendricks-Sturrup & Lu, 2019)?
5. What happens to data if the company is acquired, merges, or goes bankrupt? Are those possibilities explained and governed (Federal Trade Commission, 2025)?

6. Are we increasing discrimination risk if genetic information is leaked, and do we comply with protections such as GINA while also anticipating harms that the law may not fully prevent (U.S. Equal Employment Opportunity Commission, 2014; Panacer, 2023)?
7. Breach readiness: If an incident occurs, can we notify users in a quick and accurate manner, and can we enable them to take protective actions (Office of the Privacy Commissioner of Canada, 2025)?

These questions should be asked repeatedly: at initial design, before launch, during major feature changes, and during organizational transitions.

What Can the Software Community Do to Support Coping with Ethical Issues?

The individual software developers are often pressured to ship faster, reduce conflicts with business stakeholders and management, increase engagement, or monetize data. To reduce this burden, the software community can share norms and provide practical support. First, the community can uphold professional standards as living guidance rather than static documents. The ACM Code of Ethics is a cornerstone of computing ideals; it frames obligations, such as avoiding harm respect privacy. This can be used by developers to justify ethical engineering decisions when short-term business incentives conflict with safety (ACM, 2018). Second, the community can support privacy engineering and security engineering as a core component of software development competencies. For example, credential stuffing and automated scraping are known and predictable threats. In the 23andMe case, the investigation found that distortions in login success/failure ratios can be indicators of credential-stuffing attacks. In other words, credential-stuffing attacks are foreseeable risks; therefore, organizations should implement detection and response plans (Office of the Privacy Commissioner of Canada, 2025).

Additionally, sharing implementation patterns helps developers to adopt best practices; patterns such as rate limiting, MFA enforcement strategies, bot detection, and secure logging. Third, the community can expand education and tooling for developers. Tools such as clear checklists and frameworks like the NIST Privacy Framework provide developers a shared vocabulary for privacy risk management and accountability (NIST, 2020). Furthermore, education programs such as those provided by universities, organizations, and industry training programs should treat privacy risk as part of system design, not a legal afterthought. Fourth, the software community can support the development of ethical processes and protect whistleblowers. This can be implemented using mechanisms such as those described by the ACM, which emphasize professional responsibility and taking action when harm is likely. Finally, the community can create a safe, encouraging, and transparent environment where developers can learn from their own and others' incidents. Post-incident transparency, that is, transparency about what happened, what changed, and what controls failed, can help developers and other organizations avoid repeating the same mistakes.

Feasible Remedies and Critical Evaluation (Are They Doable?)

Remedies must be evaluated not only for technical correctness but for feasibility: cost, operational complexity, user experience impact, and organizational ownership. This section maps identified lapses to remedies and evaluates feasibility in realistic terms, see the table below.

Table 1

Remedy map (lapse → remedy → feasibility)

Ethical lapse	Concrete remedy	Primary owner(s)	Friction / tradeoff	Feasibility
Credential-stuffing exposure	Mandatory MFA for all accounts.	Security engineering + product	Added login friction; support burden	High

Credential-stuffing exposure	Rate limiting, bot detection, credential-stuffing monitoring using success/failure ratio anomalies	Security engineering + SRE	Engineering time; tuning false positives	High
Social feature "blast radius"	Reduce exposure by design.	Product + privacy engineering	May reduce feature utility; potential engagement decline	Medium–High
Weak detection/response	Logging, alerting, and incident-response playbooks. All centralized.	SRE + security + leadership	Ongoing operational cost	High
Inadequate notifications	Breach notification quality controls.	Legal/privacy office + security	Coordination overhead; legal review time	High
Secondary use ambiguity	Consent integrates plain-language summaries; opt-in for research/third parties; easy consent withdrawal	Product + privacy office + UX	Requires cross-functional governance; possible revenue impact	Medium
Bankruptcy / transfer governance	Data transfer governance designed with restrictions, data-use covenants, and technical controls supporting deletion and isolation.	Executives + legal/privacy + security	Business constraints; may affect sale value	Medium

Note: The table describes the remedies and their primary owner(s), friction / tradeoff, feasibility.

The feasibility of these remedies varies based on their technicality and their impact on the organizational structure of a software system. Mandatory MFA and layered authentication controls are considered highly implementable, as they can be integrated across consumer accounts and can prevent access using recycled passwords. Similarly, implementing rate limiting, monitoring, and anomaly detection is standard Site Reliability Engineering (SRE) practice. In the 23andMe case, the investigation found that credential-stuffing attacks were detectable in platform logs, allowing them to be detected through monitoring (Office of the Privacy Commissioner of Canada, 2025).

Improved incident response and communication workflows are also highly feasible, requiring organizational discipline and mature security playbooks to address existing notification shortcomings. However, some remedies require more significant investment or trade-offs. For instance, creating safer product defaults for social matching features is necessary to reduce the high blast radius caused by compromised accounts. Still, it requires balancing security with the utility of the feature. Other moderately feasible actions include clarifying secondary data use through plain-language consent and establishing bankruptcy data governance to ensure privacy promises persist through organizational transitions.

Should There Be Consequences of Unethical Software? Why or Why Not?

There should be consequences for unethical software because incentives shape behavior. Without consequences, privacy and security often become optional "nice to have" features that lose priority when budgets tighten or growth slows. In the 23andMe case, regulators concluded that safeguards were not appropriate for the sensitivity and scale of the data involved and documented shortcomings in detection and notification (Office of the Privacy Commissioner of Canada, 2025). Public agencies also intervened: the FTC raised concerns about consumer data protections during bankruptcy (FTC, 2025), and state attorneys general sought to prevent genetic data from being auctioned without informed consent (New York State Office of the Attorney General, 2025). Consequences should not be purely punitive. They can be corrective and preventive: mandated security improvements, external audits, enforceable governance commitments, and limitations on secondary use until consent is strengthened. Professional consequences also matter. The ACM Code of Ethics makes clear that ethical violations are inconsistent with professional obligations and encourages action to address ethical problems (ACM, 2018). In short, accountability must exist at both organizational and professional levels.

Conclusion

The 23andMe case demonstrates why ethics is foundational, not optional, in software development. The joint regulator findings describe how a predictable attack pattern (credential stuffing) combined with platform and product design characteristics (social matching exposure and delayed detection) produced broad risks to millions of users. The bankruptcy context added a governance dimension, raising questions about stewardship, consent, and whether privacy promises remain enforceable when ownership changes. For software professionals, especially in IT, computer science, and AI, ethical practice means translating values into controls: strong authentication and monitoring, safer defaults, meaningful consent, and lifecycle governance. Frameworks such as the ACM Code of Ethics and the NIST Privacy Framework provide a disciplined way to do this work. When teams operationalize these standards, they reduce avoidable harms and strengthen public trust in data-driven systems

References

23andMe. (2023, October 6). *Addressing data security concerns – action plan* (Updated December 5, 2023). 23andMe Blog. https://blog.23andme.com/articles/addressing-data-security-concerns

ACM. (2018). *ACM code of ethics and professional conduct* [Booklet]. Association for Computing Machinery. https://www.acm.org/binaries/content/assets/about/acm-code-of-ethics-booklet.pdf

Ferguson, A. N. (2025, March 31). *Chairman Ferguson letter regarding 23andMe* [Letter]. Federal Trade Commission. https://www.ftc.gov/news-events/news/press-releases/2025/03/federal-trade-commission-chairman-andrew-n-ferguson-issues-letter-23andme-bankruptcy-impact

Hendricks-Sturrup, R. M., & Lu, C. Y. (2019). Hendricks-Sturrup, R. M., & Lu, C. Y. (2019). Direct-to-consumer genetic testing data privacy: Key concerns and recommendations based on consumer perspectives. *Journal of Personalized Medicine, 9*(2), 25. https://doi.org/10.3390/jpm9020025

NIST. (2020, January 16). *NIST privacy framework: A tool for improving privacy through enterprise risk management* (Version 1.0). National Institute of Standards and Technology. https://www.nist.gov/document/nist-privacy-frameworkv10pdf

National Library of Medicine. (2022, June 23). *How do direct-to-consumer genetic testing companies protect their customers' privacy?* MedlinePlus Genetics. https://medlineplus.gov/genetics/understanding/dtcgenetictesting/dtcprivacy/

New York State Office of the Attorney General. (2025, June 10). Attorney General James sues 23andMe to protect New Yorkers' genetic data. https://ag.ny.gov/press-release/2025/attorney-general-james-sues-23andme-protect-new-yorkers-genetic-data

Office of the Privacy Commissioner of Canada. (2025, June 20). *PIPEDA Findings #2025-001: Joint investigation into a data breach at 23andMe by the Privacy Commissioner of Canada and the UK Information Commissioner*. Office of the Privacy Commissioner of Canada https://www.priv.gc.ca/en/opc-actions-and-decisions/investigations/investigations-into-businesses/2025/pipeda-2025-001/

Panacer, K. S. (2023). Ethical issues associated with direct-to-consumer genetic testing. *Cureus, 15*(6), e39918. https://doi.org/10.7759/cureus.39918

Satija, B., & Mahatole, S. (2025, March 26). *Consumers urged to delete 23andMe data as bankruptcy sparks privacy fears*. Reuters. https://www.reuters.com/business/healthcare-pharmaceuticals/new-york-attorney-general-urges-23andme-users-delete-their-data-2025-03-25/

Equal Employment Opportunity Commission. (2014, September 9). *Fact sheet: Genetic Information Nondiscrimination Act*. U.S. Equal Employment Opportunity Commission. https://www.eeoc.gov/laws/guidance/fact-sheet-genetic-information-nondiscrimination-act